ALERTACLIMA CIDADÃO — GLOBAL SOLUTION

Coding for Security | CitizenTech | FIAP 2026 | Relatório técnico — MongoDB Atlas

1. Problema e cidadão atendido

Em cidades, ondas de calor podem prejudicar a saúde e o cotidiano. O cidadão atendido é o morador de áreas urbanas, especialmente idosos, crianças e trabalhadores expostos ao sol. O problema é a dificuldade de acompanhar a temperatura de regiões de interesse e compará-la a um limite de referência.

2. Ideia da Inspiração e evidência

A solução se alinha ao eixo de sistemas de monitoramento de eventos climáticos extremos e ondas de calor da proposta CitizenTech. A evidência pública de relevância é a orientação da Organização Mundial da Saúde sobre riscos do calor à saúde, disponível em: https://www.who.int/news-room/fact-sheets/detail/climate-change-heat-and-health. Não foi realizada entrevista nem observação de campo; a correspondência literal com a atividade do evento deve ser confirmada.

3. ODS relacionadas

ODS 3 — Saúde e Bem-Estar: apoio à prevenção de efeitos do calor. ODS 11 — Cidades e Comunidades Sustentáveis: informação local para moradores. ODS 13 — Ação Climática: conscientização e adaptação a eventos de calor.

4. Objetivo e funcionamento

O usuário cadastra bairro, cidade, latitude, longitude e limite de calor. A interface envia os dados à API Flask, que valida e grava no MongoDB Atlas. A consulta meteorológica usa a API pública Open-Meteo e compara a temperatura atual com o limite. O aviso é informativo: não é alerta oficial nem diagnóstico de saúde.

5. Escopo

O protótipo permite criar, listar, editar e excluir regiões, consultar temperatura, registrar logs e executar um serviço TCP local de status. Não oferece envio automático de mensagens, previsão de emergência, autenticação de cidadãos nem alertas oficiais.

ARQUITETURA E MODELO DE DADOS

6. Componentes e fluxo

Frontend: HTML, CSS e JavaScript. Backend: Python e Flask. Persistência: MongoDB Atlas, acessado diretamente pelo PyMongo, sem ORM. Serviço externo: Open-Meteo com requests e JSON. Auditoria: logs em arquivo. Módulo adicional: socket TCP local, porta 5051.

Fluxo: navegador → Flask → MongoDB Atlas (CRUD). Para temperatura: navegador → Flask → Open-Meteo. O serviço TCP roda em processo separado e responde ao comando STATUS.

7. Modelo de dados

Banco alertaclima, coleção alertas: id (inteiro único), bairro (texto), cidade (texto), latitude (número), longitude (número) e limite_calor (número). A coleção contadores mantém a sequência dos identificadores. Não são coletados nome, CPF, telefone ou endereço residencial.

8. Endpoints REST

Método	Rota	Resultado
GET	/api/alertas	Lista — 200
POST	/api/alertas	Criação — 201
PUT	/api/alertas/	Atualização — 200 / 404
DELETE	/api/alertas/	Exclusão — 204 / 404
GET	/api/alertas//clima	Temperatura — 200 / 502

9. Organização do projeto

Os módulos app/api.py, database.py, clima.py, validacao.py, logs.py, config.py e tcp_status.py separam responsabilidades. O arquivo run.py inicia a aplicação. requirements.txt lista dependências; .env.example mostra variáveis sem divulgar credenciais; README orienta a execução.

SEGURANÇA, TESTES E LIMITAÇÕES

10. Três ameaças e respectivas mitigações

1 — Vazamento de credenciais: string de conexão do Atlas exposta no código ou ZIP. Mitigação: usar arquivo .env excluído da entrega e usuário com readWrite restrito ao banco alertaclima, com senha forte.

2 — Entrada maliciosa ou inválida: campos manipulados e valores fora do intervalo. Mitigação: validação de tipos, tamanho, coordenadas e limites antes do acesso ao banco; uso de filtros e operações do driver PyMongo em vez de consultas montadas com texto do usuário.

3 — Falha de serviços externos: indisponibilidade do Atlas ou Open-Meteo. Mitigação: timeout de conexão, tratamento de exceções, códigos HTTP apropriados e mensagens genéricas ao cidadão.

11. Privacidade e LGPD

Coleta mínima: apenas dados de localização de regiões, não identificadores de pessoas. Credenciais ficam fora do repositório; logs registram ações e resultados sem senha. A conexão do Atlas deve restringir os IPs autorizados. O protótipo não tem autenticação de usuários, portanto deve rodar apenas localmente.

12. Validação prevista

Testes automatizados com pytest verificam validação e respostas das rotas por mocks. Testes manuais devem incluir cadastro, listagem, edição, exclusão, consulta à Open-Meteo, conferência das coleções no Atlas, arquivo de logs e comando STATUS via TCP. Os testes reais no computador do aluno ainda precisam ser executados e documentados; não se afirma funcionamento integral sem essa etapa.

13. Limitações e viabilidade

O Atlas exige internet, credenciais e IP autorizado. O projeto depende da disponibilidade da API meteorológica. Temperatura atual não equivale a previsão de emergência. Sem login e autorização por usuário, não é adequado publicar o protótipo na internet. A implantação local é viável para demonstração acadêmica.

14. Declaração de uso de IA

Ferramentas de IA generativa auxiliaram na elaboração e revisão do código, estruturação da documentação e redação do relatório. A responsabilidade pela compreensão, testes e apresentação do trabalho permanece com o aluno.